고객을 세그먼테이션하자 [프로젝트] - 신정아

11-2. 데이터 불러오기

데이터 살펴보기

- **테이블에 있는 10개의 행만 출력하기**

```
SELECT *
FROM `project-5faeab26-b699-4f50-a43.modulabs_project.data`
LIMIT 10
```

[결과 이미지]

쿼리 결과

+ 대화 만들기 결과 저장

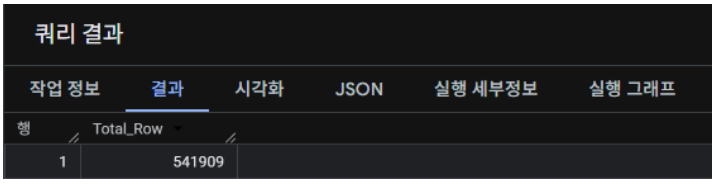
작업 정보 결과 시각화 JSON 실행 세부정보 실행 그래프

행	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country
1	536365	85123A	WHITE HANGING HEART T-LIG...	6	2010-12-01 08:26:00 UTC	2.55	17850	United Kingdom
2	536365	71053	WHITE METAL LANTERN	6	2010-12-01 08:26:00 UTC	3.39	17850	United Kingdom
3	536365	84406B	CREAM CUPID HEARTS COAT H...	8	2010-12-01 08:26:00 UTC	2.75	17850	United Kingdom
4	536365	84029G	KNITTED UNION FLAG HOT WA...	6	2010-12-01 08:26:00 UTC	3.39	17850	United Kingdom
5	536365	84029E	RED WOOLLY HOTTIE WHITE H...	6	2010-12-01 08:26:00 UTC	3.39	17850	United Kingdom
6	536365	22752	SET 7 BABUSHKA NESTING BO...	2	2010-12-01 08:26:00 UTC	7.65	17850	United Kingdom
7	536365	21730	GLASS STAR FROSTED T-LIGHT...	6	2010-12-01 08:26:00 UTC	4.25	17850	United Kingdom
8	536366	22633	HAND WARMER UNION JACK	6	2010-12-01 08:28:00 UTC	1.85	17850	United Kingdom
9	536366	22632	HAND WARMER RED POLKA DOT	6	2010-12-01 08:28:00 UTC	1.85	17850	United Kingdom
10	536367	84879	ASSORTED COLOUR BIRD ORN...	32	2010-12-01 08:34:00 UTC	1.69	13047	United Kingdom

- **전체 데이터는 몇 행으로 구성되어 있는지 확인하기**

```
SELECT COUNT(*) AS Total_Row
FROM `project-5faeab26-b699-4f50-a43.modulabs_project.data`
```

[결과 이미지]



쿼리 결과

작업 정보 결과 시각화 JSON 실행 세부정보 실행 그래프

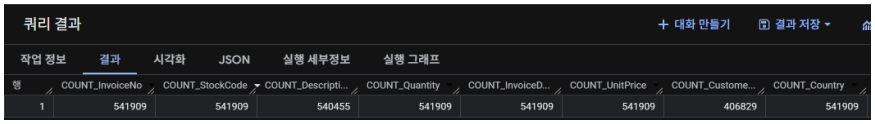
행	Total_Row
1	541909

데이터 수 세기

- **COUNT 함수를 사용해서, 각 컬럼별 데이터 포인트의 수를 세어 보기**

```
SELECT COUNT(InvoiceNo) AS COUNT_InvoiceNo,
       COUNT(StockCode) AS COUNT_StockCode,
       COUNT(Description) AS COUNT_Description,
       COUNT(Quantity) AS COUNT_Quantity,
       COUNT(InvoiceDate) AS COUNT_InvoiceDate,
       COUNT(UnitPrice) AS COUNT_UnitPrice,
       COUNT(CustomerID) AS COUNT_CustomerID,
       COUNT(Country) AS COUNT_Country
FROM `project-5faeab26-b699-4f50-a43.modulabs_project.data`
```

[결과 이미지]



행	COUNT_InvoiceNo	COUNT_StockCode	COUNT_Descripti...	COUNT_Quantity	COUNT_InvoiceD...	COUNT_UnitPrice	COUNT_Custome...	COUNT_Country
1	541909	541909	540455	541909	541909	541909	406829	541909

11-4. 데이터 전처리 방법(1): 결측치 제거

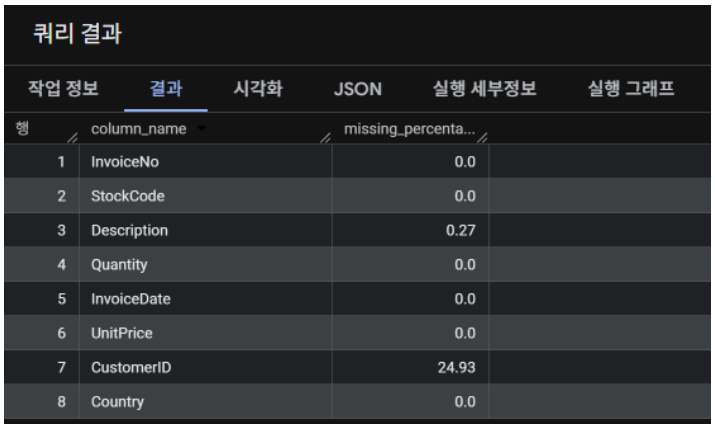
컬럼 별 누락된 값의 비율 계산

- **각 컬럼 별 누락된 값의 비율을 계산**
  - **방법1) 각 컬럼에 대해서 누락 값을 계산한 후, 계산된 누락 값을 UNION ALL을 통해 합치기**
    1) 각 컬럼에 대한 누락값 계산 : SELECT 컬럼명, 결측비율계산 FROM 테이블명
    2) 1)에서 도출한 테이블 컬럼명 column_name, missing_percentage 의 테이블을 UNION ALL 을 이용하여 세로로 합치기
    ※ UNION vs UNION ALL :
    - UNION : 테이블을 아래로, 세로로 붙이는 형태 / 컬럼명이 달라도 적용 가능 (중복 제거)
    - UNION ALL : 중복 포함하여 테이블을 세로로 합칠 때 사용

```
SELECT
    'InvoiceNo' AS column_name,
    ROUND(SUM(CASE WHEN InvoiceNo IS NULL THEN 1 ELSE 0 END) / COUNT(*) * 100, 2) AS missi
ng_percentage
FROM `project-5faeab26-b699-4f50-a43.modulabs_project.data`
UNION ALL
SELECT
    'StockCode' AS column_name,
    ROUND(SUM(CASE WHEN StockCode IS NULL THEN 1 ELSE 0 END) / COUNT(*) * 100, 2) AS missi
ng_percentage
FROM `project-5faeab26-b699-4f50-a43.modulabs_project.data`
UNION ALL
SELECT
    'Description' AS column_name,
    ROUND(SUM(CASE WHEN Description IS NULL THEN 1 ELSE 0 END) / COUNT(*) * 100, 2) AS mis
sing_percentage
FROM `project-5faeab26-b699-4f50-a43.modulabs_project.data`
UNION ALL
SELECT
    'Quantity' AS column_name,
    ROUND(SUM(CASE WHEN Quantity IS NULL THEN 1 ELSE 0 END) / COUNT(*) * 100, 2) AS missin
g_percentage
FROM `project-5faeab26-b699-4f50-a43.modulabs_project.data`
UNION ALL
SELECT
    'InvoiceDate' AS column_name,
    ROUND(SUM(CASE WHEN InvoiceDate IS NULL THEN 1 ELSE 0 END) / COUNT(*) * 100, 2) AS mis
sing_percentage
FROM `project-5faeab26-b699-4f50-a43.modulabs_project.data`
UNION ALL
SELECT
    'UnitPrice' AS column_name,
    ROUND(SUM(CASE WHEN UnitPrice IS NULL THEN 1 ELSE 0 END) / COUNT(*) * 100, 2) AS missi
ng_percentage
FROM `project-5faeab26-b699-4f50-a43.modulabs_project.data`
UNION ALL
SELECT
    'CustomerID' AS column_name,
    ROUND(SUM(CASE WHEN CustomerID IS NULL THEN 1 ELSE 0 END) / COUNT(*) * 100, 2) AS miss
ing_percentage
FROM `project-5faeab26-b699-4f50-a43.modulabs_project.data`
```

```
UNION ALL
SELECT
    'Country' AS column_name,
    ROUND(SUM(CASE WHEN Country IS NULL THEN 1 ELSE 0 END) / COUNT(*) * 100, 2) AS missing
_percentage
FROM `project-5faeab26-b699-4f50-a43.modulabs_project.data`
```

[결과 이미지 1]



쿼리 결과

작업 정보 | 결과 | 시각화 | JSON | 실행 세부정보 | 실행 그래프

행	column_name	missing_percenta...
1	InvoiceNo	0.0
2	StockCode	0.0
3	Description	0.27
4	Quantity	0.0
5	InvoiceDate	0.0
6	UnitPrice	0.0
7	CustomerID	24.93
8	Country	0.0

- **방법2) 각 컬럼에 대해서 누락 값을 계산한 후, 계산된 누락 값을 UNION ALL을 통해 합쳐서 새로운 테이블 생성, 이후 새로운 테이블에서 컬럼명, 결측률을 계산하여 추출**
  1) 각 컬럼에 대한 누락값 계산 : SELECT 컬럼명, COUNT(컬럼명), COUNT(*)
  2) 1)에서 도출한 테이블 컬럼명 column_name, 해당 컬럼 데이터수, 전체 행수로 구성된 테이블을 UNION ALL 을 이용하여 세로로 합친 후, 새로운 테이블 생성
  3) SELECT와 산술연산을 이용하여 결측율 계산,
  4) FROM 뒤 테이블명은 앞선 (2) 단계에서 생성한 쿼리 이용

```
SELECT column_name, ROUND((total - column_value) / total * 100, 2)
FROM (
     SELECT 'InvoiceNo' AS column_name, COUNT(InvoiceNo) AS column_value, COUNT(*) AS tota
l
     FROM `project-5faeab26-b699-4f50-a43.modulabs_project.data`
     UNION ALL
     SELECT 'StockCode' AS column_name, COUNT(StockCode) AS column_value, COUNT(*) AS tota
l
     FROM `project-5faeab26-b699-4f50-a43.modulabs_project.data`
     UNION ALL
     SELECT 'Description' AS column_name, COUNT(Description) AS column_value, COUNT(*) AS
total
     FROM `project-5faeab26-b699-4f50-a43.modulabs_project.data`
     UNION ALL
     SELECT 'Quantity' AS column_name, COUNT(Quantity) AS column_value, COUNT(*) AS total
     FROM `project-5faeab26-b699-4f50-a43.modulabs_project.data`
     UNION ALL
     SELECT 'InvoiceDate' AS column_name, COUNT(InvoiceDate) AS column_value, COUNT(*) AS
total
     FROM `project-5faeab26-b699-4f50-a43.modulabs_project.data`
     UNION ALL
     SELECT 'UnitPrice' AS column_name, COUNT(UnitPrice) AS column_value, COUNT(*) AS tota
l
     FROM `project-5faeab26-b699-4f50-a43.modulabs_project.data`
     UNION ALL
     SELECT 'CustomerID' AS column_name, COUNT(CustomerID) AS column_value, COUNT(*) AS to
tal
     FROM `project-5faeab26-b699-4f50-a43.modulabs_project.data`
```

```
    UNION ALL
    SELECT 'Country' AS column_name, COUNT(Country) AS column_value, COUNT(*) AS total
    FROM `project-5faeab26-b699-4f50-a43.modulabs_project.data`)
    AS column_data
```

결측치 처리 전략

- `StockCode = '85123A'` 의 `Description` 을 추출하는 쿼리문을 작성하기

```
SELECT DISTINCT Description
FROM `project-5faeab26-b699-4f50-a43.modulabs_project.data`
WHERE StockCode = '85123A'
```

[결과 이미지]

행	Description
1	WHITE HANGING HEART T-LIGHT HOLDER
2	?
3	wrongly marked carton 22804
4	CREAM HANGING HEART T-LIGHT HOLDER

- `StockCode = '85123A'` 의 `Description` 을 추출(해당 데이터의 고유값 및 고유값 개수 확인) 쿼리문

```
SELECT Description, COUNT(*) AS cnt
FROM `project-5faeab26-b699-4f50-a43.modulabs_project.data`
WHERE StockCode = '85123A'
GROUP BY 1
ORDER BY cnt DESC
```

[결과 이미지]

행	Description	cnt
1	WHITE HANGING HEART T-LIGHT HOLDER	2302
2	CREAM HANGING HEART T-LIGHT HOLDER	9
3	wrongly marked carton 22804	1
4	?	1

결측치 처리

- **DELETE 구문을 사용하며, WHERE 절을 통해 데이터를 제거할 조건을 제시**

```
# DELETE 구문 사용 전, [SELECT * FROM 테이블명 WHERE 조건]으로
# 삭제할 데이터를 확인 후, DELETE FROM 테이블명 WHERE 조건 적용
DELETE FROM `project-5faeab26-b699-4f50-a43.modulabs_project.data`
WHERE Description IS NULL OR CustomerID IS NULL
```

[결과 이미지]

작업 정보 | 결과 | 실행 세부정보 | 실행 그래프

이 문으로 data의 행 135,080개가 삭제되었습니다.

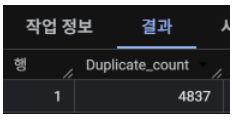
11-5. 데이터 전처리(2): 중복값 처리

중복값 확인

- **중복된 행의 수를 세어보기**
  - **8개의 컬럼에 그룹 함수를 적용한 후, COUNT가 1보다 큰 데이터를 세어보기**

```
SELECT COUNT(*) AS Duplicate_count
FROM(
SELECT *,
       COUNT(*) AS cnt
FROM `project-5faeab26-b699-4f50-a43.modulabs_project.data`
GROUP BY InvoiceNo, StockCode, Description, Quantity, InvoiceDate, UnitPrice, CustomerID,
Country
HAVING cnt > 1)
```

[결과 이미지 - 중복된 행 수 계산]



  - **8개의 컬럼에 그룹 함수를 적용한 후, COUNT가 1보다 큰 데이터 확인**

```
SELECT InvoiceNo, StockCode, Description, Quantity, InvoiceDate, UnitPrice, CustomerID, Co
untry,
       COUNT(*) AS cnt
FROM `project-5faeab26-b699-4f50-a43.modulabs_project.data`
GROUP BY 1, 2, 3, 4, 5, 6, 7, 8
HAVING cnt > 1
```

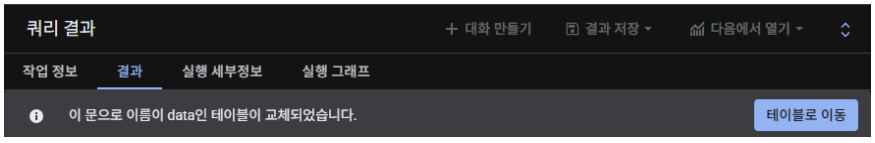
[결과 이미지 - FROM 속 테이블]

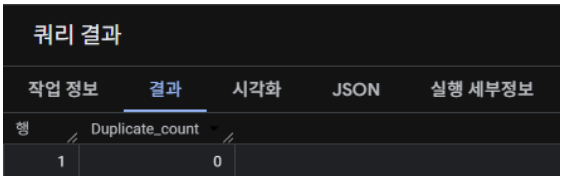
중복값 처리

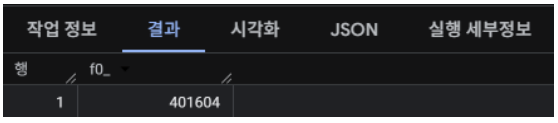
- **중복값을 제거하는 쿼리문 작성하기**
  - **CREATE OR REPLACE TABLE 구문을 활용하여 모든 컬럼(*)을 DISTINCT 한 데이터로 업데이트**

```
CREATE OR REPLACE TABLE `project-5faeab26-b699-4f50-a43.modulabs_project.data`
AS
SELECT DISTINCT *
FROM `project-5faeab26-b699-4f50-a43.modulabs_project.data`
```

[결과 이미지]







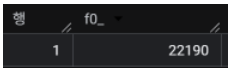
11-6. 데이터 전처리(3): 오류값 처리

`InvoiceNo` 살펴보기

- **고유(unique)한 `InvoiceNo`의 개수를 출력하기**

```
SELECT COUNT(*)
FROM (
SELECT InvoiceNo, COUNT(*) AS cnt
FROM `project-5faeab26-b699-4f50-a43.modulabs_project.data`
GROUP BY 1
ORDER BY cnt DESC)
```

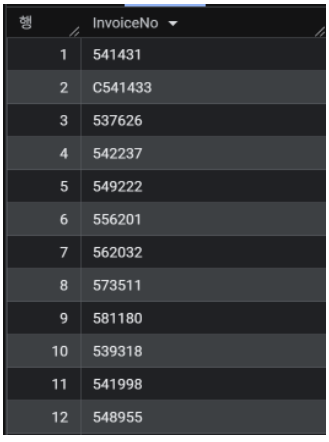
[결과 이미지]



- **고유한 `InvoiceNo`를 앞에서부터 100개를 출력하기**

```
SELECT DISTINCT InvoiceNo
FROM `project-5faeab26-b699-4f50-a43.modulabs_project.data`
LIMIT 100
```

[결과 이미지]



행	InvoiceNo
1	541431
2	C541433
3	537626
4	542237
5	549222
6	556201
7	562032
8	573511
9	581180
10	539318
11	541998
12	548955

- **`InvoiceNo`가 'C'로 시작하는 행을 필터링 할 수 있는 쿼리문을 작성하기 (100행까지만 출력)**

```
SELECT *
FROM `project-5faeab26-b699-4f50-a43.modulabs_project.data`
WHERE InvoiceNo LIKE 'C%'
LIMIT 100
```

[결과 이미지]

행	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country
1	C541433	23166	MEDIUM CERAMIC TOP STORA...	-74215	2011-01-18 10:17:00 UTC	1.04	12346	United Kingdom
2	C545329	M	Manual	-1	2011-03-01 15:47:00 UTC	183.75	12352	Norway
3	C545329	M	Manual	-1	2011-03-01 15:47:00 UTC	280.05	12352	Norway
4	C545330	M	Manual	-1	2011-03-01 15:49:00 UTC	376.5	12352	Norway
5	C547388	22784	LANTERN CREAM GAZEBO	-3	2011-03-22 16:07:00 UTC	4.95	12352	Norway
6	C547388	84050	PINK HEART SHAPE EGG FRYIN...	-12	2011-03-22 16:07:00 UTC	1.65	12352	Norway
7	C547388	22413	METAL SIGN TAKE IT OR LEAVE...	-6	2011-03-22 16:07:00 UTC	2.95	12352	Norway
8	C547388	22701	PINK DOG BOWL	-6	2011-03-22 16:07:00 UTC	2.95	12352	Norway
9	C547388	22645	CERAMIC HEART FAIRY CAKE ...	-12	2011-03-22 16:07:00 UTC	1.45	12352	Norway

- **구매 건 상태가 `Canceled` 인 데이터의 비율(%) - 소수점 첫번째 자리까지**

```
SELECT ROUND(SUM(CASE WHEN InvoiceNo LIKE 'C%' THEN 1 ELSE 0 END)/COUNT(*)*100,1)
FROM `project-5faeab26-b699-4f50-a43.modulabs_project.data`
```

[결과 이미지]

행	f0_
1	2.2

`StockCode` 살펴보기

- **고유한 `StockCode`의 개수를 출력하기**

```
SELECT COUNT(*)
FROM(
SELECT StockCode, COUNT(*) AS cnt
FROM `project-5faeab26-b699-4f50-a43.modulabs_project.data`
GROUP BY 1)
```

[결과 이미지]

행	f0_
1	3684

- **어떤 제품이 가장 많이 판매되었는지 보기 위하여 `StockCode` 별 등장 빈도를 출력하기**
  - 상위 10개의 제품들을 출력하기

```
SELECT StockCode, COUNT(*) AS sell_cnt
FROM `project-5faeab26-b699-4f50-a43.modulabs_project.data`
GROUP BY 1
ORDER BY sell_cnt DESC
LIMIT 10
```

[결과 이미지]

행	StockCode	sell_cnt
1	85123A	2065
2	22423	1894
3	85099B	1659
4	47566	1409
5	84879	1405
6	20725	1346
7	22720	1224
8	POST	1196
9	22197	1110
10	23203	1108

※ 특징 : StockCode는 5~6자리의 숫자와 문자로 구성되어 있다.
'POST'라고 적힌 데이터의 Description을 확인해본 결과, POSTAGE로 상품이 아닌 배송비 형태를 코드로 남김 것으로 볼 수 있다.

행	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country
1	539318	POST	POSTAGE	6	2010-12-16 19:09:00 UTC	40.0	12348	Finland
2	541998	POST	POSTAGE	1	2011-01-25 10:42:00 UTC	40.0	12348	Finland
3	548955	POST	POSTAGE	1	2011-04-05 10:47:00 UTC	40.0	12348	Finland
4	568172	POST	POSTAGE	1	2011-09-25 13:13:00 UTC	40.0	12348	Finland
5	577609	POST	POSTAGE	1	2011-11-21 09:51:00 UTC	300.0	12349	Italy
6	543037	POST	POSTAGE	1	2011-02-02 16:01:00 UTC	40.0	12350	Norway
7	545323	POST	POSTAGE	1	2011-03-01 14:57:00 UTC	40.0	12352	Norway
8	547390	POST	POSTAGE	1	2011-03-22 16:08:00 UTC	40.0	12352	Norway
9	567505	POST	POSTAGE	2	2011-09-20 14:34:00 UTC	40.0	12352	Norway
10	568699	POST	POSTAGE	1	2011-09-28 14:58:00 UTC	40.0	12352	Norway

- `StockCode` **의 컬럼에 있던 값 중에서 숫자를 제외한 문자만 남기고 문자가 몇 자리 수 인지 세고**
  - **숫자가 0~1개인 값**들에는 어떤 코드들이 들어가 있는지 출력하기

```
SELECT DISTINCT StockCode, number_count
FROM(
    SELECT StockCode,
           LENGTH(StockCode) - LENGTH(REGEXP_REPLACE(StockCode, r'[0-9]', '')) AS number_c
ount
    FROM `project-5faeab26-b699-4f50-a43.modulabs_project.data`
)
WHERE number_count = 0 OR number_count = 1
```

[결과 이미지]

행	StockCode	number_count
1	POST	0
2	M	0
3	C2	1
4	D	0
5	BANK CHARGES	0
6	PADS	0
7	DOT	0
8	CRUK	0

- `StockCode` **의 컬럼에 있던 값 중에서 숫자를 제외한 문자만 남기고 문자가 몇 자리 수 인지 세고**
  - **숫자가 0~1개인 값들을 가지고 있는 데이터 수는 전체 데이터 수 대비 몇 퍼센트**인지 구하기 (소수점 두 번째 자리까지)

```
SELECT ROUND(
    SUM(CASE WHEN number_count<=1 THEN 1 ELSE 0 END)/COUNT(*) * 100 ,2) AS abnormal_rate
FROM(
    SELECT StockCode,
           LENGTH(StockCode) - LENGTH(REGEXP_REPLACE(StockCode, r'[0-9]', '')) AS number_c
ount
    FROM `project-5faeab26-b699-4f50-a43.modulabs_project.data`)
```

[결과 이미지]

행	abnormal_rate
1	0.48

- **제품과 관련되지 않은 거래 기록을 제거하기**

```
DELETE FROM `project-5faeab26-b699-4f50-a43.modulabs_project.data`
WHERE StockCode IN (
    SELECT DISTINCT StockCode
    FROM(
```

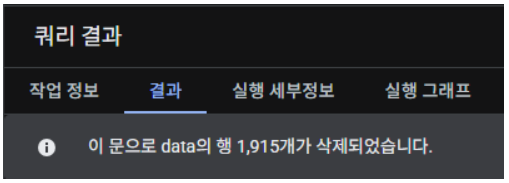
```
        SELECT StockCode,
               LENGTH(StockCode) - LENGTH(REGEXP_REPLACE(StockCode, r'[0-9]', '')) AS num
ber_count
        FROM `project-5faeab26-b699-4f50-a43.modulabs_project.data`
    )
    WHERE number_count = 0 OR number_count = 1)
```

[결과 이미지 _ 삭제 대상 데이터 확인]

행	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country
1	539318	POST	POSTAGE	6	2010-12-16 19:09:00 UTC	40.0	12348	Finland
2	541998	POST	POSTAGE	1	2011-01-25 10:42:00 UTC	40.0	12348	Finland
3	548955	POST	POSTAGE	1	2011-04-05 10:47:00 UTC	40.0	12348	Finland
4	568172	POST	POSTAGE	1	2011-09-25 13:13:00 UTC	40.0	12348	Finland
5	577609	POST	POSTAGE	1	2011-11-21 09:51:00 UTC	300.0	12349	Italy
6	543037	POST	POSTAGE	1	2011-02-02 16:01:00 UTC	40.0	12350	Norway
7	545323	POST	POSTAGE	1	2011-03-01 14:57:00 UTC	40.0	12352	Norway
8	545332	M	Manual	1	2011-03-01 15:52:00 UTC	376.5	12352	Norway
9	545332	M	Manual	1	2011-03-01 15:52:00 UTC	280.05	12352	Norway
10	545332	M	Manual	1	2011-03-01 15:52:00 UTC	183.75	12352	Norway
11	547390	POST	POSTAGE	1	2011-03-22 16:08:00 UTC	40.0	12352	Norway
12	567505	POST	POSTAGE	2	2011-09-20 14:34:00 UTC	40.0	12352	Norway
13	568699	POST	POSTAGE	1	2011-09-28 14:58:00 UTC	40.0	12352	Norway
14	574275	POST	POSTAGE	2	2011-11-03 14:37:00 UTC	40.0	12352	Norway

페이지당 결과 수: 50 1 – 50 (전체 1915행)

[결과 이미지]



`Description` 살펴보기

- **고유한 Description 별 출현 빈도를 계산하고 상위 30개를 출력하기**

```
SELECT Description, COUNT(*) AS description_cnt
FROM `project-5faeab26-b699-4f50-a43.modulabs_project.data`
GROUP BY 1
ORDER BY description_cnt DESC
LIMIT 30
```

[결과 이미지]

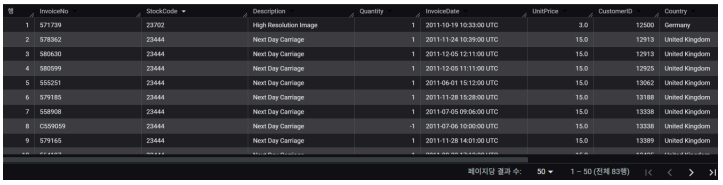
행	Description	description_cnt
1	WHITE HANGING HEART T-LIG...	2058
2	REGENCY CAKESTAND 3 TIER	1894
3	JUMBO BAG RED RETROSPOT	1659
4	PARTY BUNTING	1409
5	ASSORTED COLOUR BIRD ORN...	1405
6	LUNCH BAG RED RETROSPOT	1345
7	SET OF 3 CAKE TINS PANTRY D...	1224
8	LUNCH BAG BLACK SKULL.	1099
9	PACK OF 72 RETROSPOT CAKE ...	1062
10	SPOTTY BUNTING	1026

페이지당 결과 수: 50 1 – 30 (전체 30행)

- **서비스 관련 정보를 포함하는 행들을 제거하기**

```
# 서비스 관련정보를 포함하는 행 확인
SELECT *
FROM `project-5faeab26-b699-4f50-a43.modulabs_project.data`
WHERE Description LIKE 'Next Day%' OR Description  LIKE '%High Resolution%'
```

[결과 이미지-삭제할 데이터 확인]

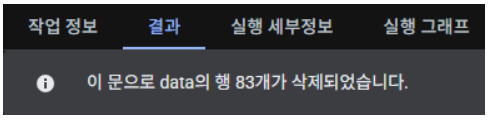


행	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country
1	571739	23702	High Resolution Image	1	2011-10-19 10:33:00 UTC	3.0	12500	Germany
2	578362	23444	Next Day Carriage	1	2011-11-24 10:39:00 UTC	15.0	12913	United Kingdom
3	580630	23444	Next Day Carriage	1	2011-12-05 12:11:00 UTC	15.0	12913	United Kingdom
4	580599	23444	Next Day Carriage	1	2011-12-05 11:11:00 UTC	15.0	12925	United Kingdom
5	555251	23444	Next Day Carriage	1	2011-06-01 15:12:00 UTC	15.0	13062	United Kingdom
6	579185	23444	Next Day Carriage	1	2011-11-28 15:28:00 UTC	15.0	13188	United Kingdom
7	558908	23444	Next Day Carriage	1	2011-07-05 09:06:00 UTC	15.0	13338	United Kingdom
8	C559059	23444	Next Day Carriage	-1	2011-07-06 10:00:00 UTC	15.0	13338	United Kingdom
9	579165	23444	Next Day Carriage	1	2011-11-28 14:01:00 UTC	15.0	13389	United Kingdom

페이지당 결과 수: 50 1 – 50 (전체 83행)

```
DELETE
FROM `project-5faeab26-b699-4f50-a43.modulabs_project.data`
WHERE Description LIKE 'Next Day%' OR Description  LIKE '%High Resolution%'
```

[결과 이미지]



- **대소문자를 혼합하고 있는 데이터를 대문자로 표준화 하기**
  ※ * EXCEPT(컬럼명) : 테이블의 모든 컬럼을 가져오되, 기재된 컬럼은 제외하라는 명령어
  (제거하고자 하는 컬럼이 유지하고자 하는 컬럼수보다 적을 때 사용)
  ※ UPPER(컬럼명) : 모든 소문자를 대문자로 변환

```
# Description을 제외한 나머지 컬럼을 추출
# Description 컬럼의 데이터는 대문자로 변환
CREATE OR REPLACE TABLE `project-5faeab26-b699-4f50-a43.modulabs_project.data`
SELECT
  * EXCEPT (Description),
  UPPER(Description) AS Description
FROM `project-5faeab26-b699-4f50-a43.modulabs_project.data`
```

[결과 이미지]

작업 정보 | 결과 | 시각화 | JSON | 실행 세부정보 | 실행 그래프

행	InvoiceNo	StockCode	Quantity	InvoiceDate	UnitPrice	CustomerID	Country	Description
1	541431	23166	74215	2011-01-18 10:01:00 UTC	1.04	12346	United Kingdom	MEDIUM CERAMIC TOP STORAGE JAR
2	C541433	23166	-74215	2011-01-18 10:17:00 UTC	1.04	12346	United Kingdom	MEDIUM CERAMIC TOP STORAGE JAR
3	537626	22773	12	2010-12-07 14:57:00 UTC	1.25	12347	Iceland	GREEN DRAWER KNOB ACRYLIC EDWARDIAN
4	537626	22775	12	2010-12-07 14:57:00 UTC	1.25	12347	Iceland	PURPLE DRAWERKNOB ACRYLIC EDWARDIAN
5	537626	84997C	6	2010-12-07 14:57:00 UTC	3.75	12347	Iceland	BLUE 3 PIECE POLKADOT CUTLERY SET
6	537626	22494	12	2010-12-07 14:57:00 UTC	1.25	12347	Iceland	EMERGENCY FIRST AID TIN
7	537626	84997D	6	2010-12-07 14:57:00 UTC	3.75	12347	Iceland	PINK 3 PIECE POLKADOT CUTLERY SET
8	537626	22212	6	2010-12-07 14:57:00 UTC	2.1	12347	Iceland	FOUR HOOK WHITE LOVEBIRDS

페이지당 결과 수: 50 1 – 50 (전체 399606행)

`UnitPrice` 살펴보기

- **`UnitPrice`의 최솟값, 최댓값, 평균을 구하기**

```
SELECT MIN(UnitPrice) AS min_price, MAX(UnitPrice) AS max_price, AVG(UnitPrice) AS avg_pri
ce
FROM `project-5faeab26-b699-4f50-a43.modulabs_project.data`
```

[결과 이미지]

행	min_price	max_price	avg_price
1	0.0	649.5	2.904956757406...

- **단가가 0원인 거래의 개수, 구매 수량(`Quantity`)의 최솟값, 최댓값, 평균 구하기**

```
SELECT COUNT(Quantity) AS cnt_quantity, MIN(Quantity) AS min_quantity, MAX(Quantity) AS ma
x_quantity, AVG(Quantity) AS avg_quantity
```

```
FROM `project-5faeab26-b699-4f50-a43.modulabs_project.data`
WHERE UnitPrice = 0
```

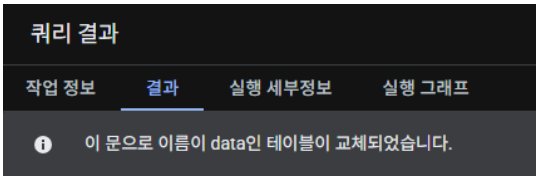
[결과 이미지]

행	cnt_quantity	min_quantity	max_quantity	avg_quantity
1	33	1	12540	420.5151515151...

- `UnitPrice = 0` **를 제거하고 일관된 데이터셋을 유지하기**
  데이터 제거 방법 : DELETE가 아닌 CREATE OR REPLACE TABLE 사용
  ※ DELETE의 경우, WHERE UnitPrice = 0 으로 필터링하여 해당 데이터를 삭제
  ※ CREATE OR REPLACE TABLE (테이블명) AS의 경우, 기존 테이블을 대체하는 명령어로, UnitPrice ≠0 인 데이터를 저장

```
CREATE OR REPLACE TABLE `project-5faeab26-b699-4f50-a43.modulabs_project.data` AS
SELECT *
FROM `project-5faeab26-b699-4f50-a43.modulabs_project.data`
WHERE UnitPrice != 0
```

[결과 이미지]



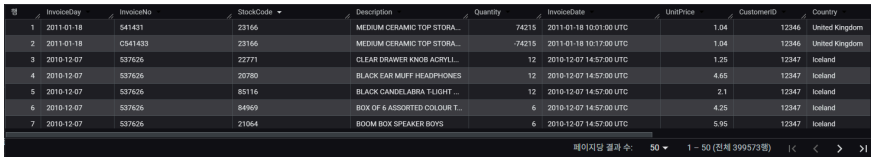
11-7. RFM 스코어

Recency

- `InvoiceDate` **컬럼을 연월일 자료형으로 변경하기**

```
SELECT DATE(InvoiceDate) AS InvoiceDay, *
FROM `project-5faeab26-b699-4f50-a43.modulabs_project.data`
```

[결과 이미지]



- **가장 최근 구매 일자를 MAX() 함수로 찾아보기**

```
# MAX() OVER() : 윈도우 함수 OVER()에 별도의 기준을 입력하지 않았을 경우,
# 전체 테이블을 하나의 그룹으로 보고 MAX 계산
SELECT
      MAX(DATE(InvoiceDate)) OVER() AS most_recent_date,
      DATE(InvoiceDate) AS InvoiceDay,
      *
FROM `project-5faeab26-b699-4f50-a43.modulabs_project.data`
```

[결과 이미지]

행	most_recent_date	InvoiceDay	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country
1	2011-12-09	2011-01-18	541431	23166	MEDIUM CERAMIC TOP STORA...	74215	2011-01-18 10:01:00 UTC	1.04	12346	United Kingdom
2	2011-12-09	2011-01-18	C541433	23166	MEDIUM CERAMIC TOP STORA...	-74215	2011-01-18 10:17:00 UTC	1.04	12346	United Kingdom
3	2011-12-09	2010-12-07	537626	22771	CLEAR DRAWER KNOB ACRYLI...	12	2010-12-07 14:57:00 UTC	1.25	12347	Iceland
4	2011-12-09	2010-12-07	537626	20780	BLACK EAR MUFF HEADPHONES	12	2010-12-07 14:57:00 UTC	4.65	12347	Iceland
5	2011-12-09	2010-12-07	537626	85116	BLACK CANDELABRA T-LIGHT ...	12	2010-12-07 14:57:00 UTC	2.1	12347	Iceland
6	2011-12-09	2010-12-07	537626	84969	BOX OF 6 ASSORTED COLOUR T...	6	2010-12-07 14:57:00 UTC	4.25	12347	Iceland
7	2011-12-09	2010-12-07	537626	21064	BOOM BOX SPEAKER BOYS	6	2010-12-07 14:57:00 UTC	5.95	12347	Iceland

페이지당 결과 수: 50 1 – 50 (전체 399573행)

```
# 집계함수 MAX() OVER(그룹설정 기준)
# MAX(DATE(InvoiceDate)) OVER(PARTITION BY InvoiceNo ORDER BY InvoiceNo DESC)
# OVER 다음 기준에 따라 MAX 계산,
# 계산된 결과를 가장 최근일 순서로 나열하기 위한 추가 쿼리문 작성 필요
SELECT
      MAX(DATE(InvoiceDate)) OVER(PARTITION BY InvoiceNo ORDER BY InvoiceNo DESC) AS most_
recent_date,
      DATE(InvoiceDate) AS InvoiceDay,
      *
FROM `project-5faeab26-b699-4f50-a43.modulabs_project.data`
ORDER BY most_recent_date DESC
```

[결과 이미지]

행	most_recent_date	InvoiceDay	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country
1	2011-12-09	2011-12-09	581587	23256	CHILDRENS CUTLERY SPACEB...	4	2011-12-09 12:50:00 UTC	4.15	12680	France
2	2011-12-09	2011-12-09	581514	22075	6 RIBBONS ELEGANT CHRISTM...	24	2011-12-09 11:20:00 UTC	0.39	17754	United Kingdom
3	2011-12-09	2011-12-09	C581568	21258	VICTORIAN SEWING BOX LARGE	-5	2011-12-09 11:57:00 UTC	10.95	15311	United Kingdom
4	2011-12-09	2011-12-09	C581490	23144	ZINC T-LIGHT HOLDER STARS S...	-11	2011-12-09 09:57:00 UTC	0.83	14397	United Kingdom
5	2011-12-09	2011-12-09	C581490	22178	VICTORIAN GLASS HANGING T-...	-12	2011-12-09 09:57:00 UTC	1.95	14397	United Kingdom
6	2011-12-09	2011-12-09	C581484	23843	PAPER CRAFT , LITTLE BIRDIE	-80995	2011-12-09 09:27:00 UTC	2.08	16446	United Kingdom
7	2011-12-09	2011-12-09	C581569	84978	HANGING HEART JAR T-LIGHT ...	-1	2011-12-09 11:58:00 UTC	1.25	17315	United Kingdom

페이지당 결과 수: 50 1 – 50 (전체 399573행)

- **유저 별로 가장 큰 InvoiceDay를 찾아서 가장 최근 구매일로 저장하기**

```
SELECT
     CustomerID,
     MAX(DATE(InvoiceDate)) AS InvoiceDay
FROM `project-5faeab26-b699-4f50-a43.modulabs_project.data`
GROUP BY 1
```

[결과 이미지]

행	CustomerID	InvoiceDay
1	12346	2011-01-18
2	12347	2011-12-07
3	12348	2011-09-25
4	12349	2011-11-21
5	12350	2011-02-02
6	12352	2011-11-03
7	12353	2011-05-19
8	12354	2011-04-21
9	12355	2011-05-09
10	12356	2011-11-17

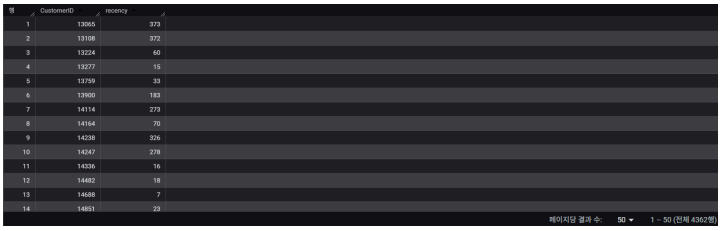
페이지당 결과 수: 50 1 – 50 (전체 4362행)

- **가장 최근 일자(`most_recent_date`)와 유저별 마지막 구매일(`InvoiceDay`)간의 차이를 계산하기**

```
SELECT
    CustomerID,
    EXTRACT(DAY FROM MAX(InvoiceDay) OVER() - InvoiceDay) AS recency
```

```
FROM(
    SELECT
        CustomerID,
        MAX(DATE(InvoiceDate)) AS InvoiceDay
    FROM `project-5faeab26-b699-4f50-a43.modulabs_project.data`
    GROUP BY 1
);
```

[결과 이미지]



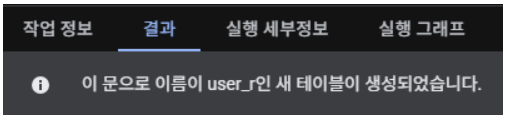
행	CustomerID	recency
1	13065	373
2	13108	372
3	13224	60
4	13277	15
5	13759	33
6	13900	183
7	14114	273
8	14164	70
9	14238	326
10	14247	278
11	14336	16
12	14482	18
13	14688	7
14	14851	23

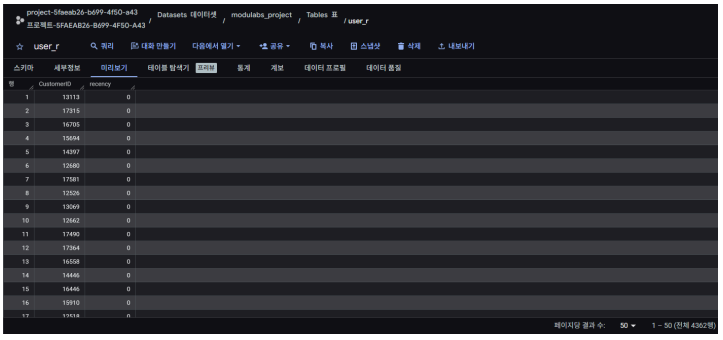
페이지당 결과 수: 50 1 – 50 (전체 4362행)

- **최종 데이터 셋에 필요한 데이터들을 각각 정제해서 이어붙이고 지금까지의 결과를 `user_r` 이라는 이름의 테이블로 저장하기**

```
CREATE OR REPLACE TABLE `project-5faeab26-b699-4f50-a43.modulabs_project.user_r` AS
SELECT
    CustomerID,
    EXTRACT(DAY FROM MAX(InvoiceDay) OVER() - InvoiceDay) AS recency
FROM(
    SELECT
        CustomerID,
        MAX(DATE(InvoiceDate)) AS InvoiceDay
    FROM `project-5faeab26-b699-4f50-a43.modulabs_project.data`
    GROUP BY 1
);
```

[결과 이미지]





행	CustomerID	recency
1	13113	0
2	17315	0
3	16705	0
4	13694	0
5	14397	0
6	12680	0
7	17581	0
8	12526	0
9	13069	0
10	13662	0
11	17490	0
12	17364	0
13	16558	0
14	14646	0
15	16446	0
16	15910	0

페이지당 결과 수: 50 1 – 50 (전체 4362행)

Frequency

- **고객마다 고유한 InvoiceNo의 수를 세어보기**

```
SELECT
    CustomerID,
    COUNT(DISTINCT InvoiceNo) AS purchase_cnt
FROM `project-5faeab26-b699-4f50-a43.modulabs_project.data`
GROUP BY 1;
```

[결과 이미지]

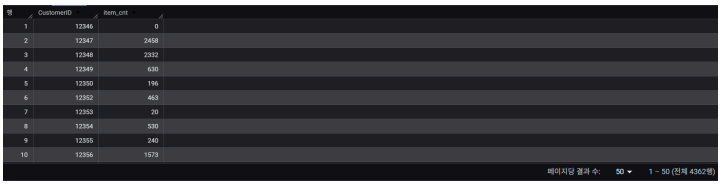
행	CustomerID	purchase_cnt
1	12346	2
2	12347	7
3	12348	4
4	12349	1
5	12350	1
6	12352	8
7	12353	1

페이지당 결과 수: 50 1 – 50 (전체 4362행)

- **각 고객 별로 구매한 아이템의 총 수량 더하기**

```
SELECT
    CustomerID,
    SUM(Quantity) AS item_cnt
FROM `project-5faeab26-b699-4f50-a43.modulabs_project.data`
GROUP BY 1;
```

[결과 이미지]



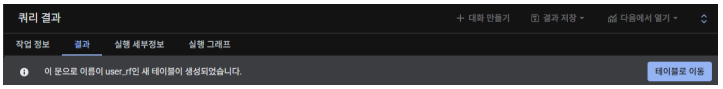
행	CustomerID	item_cnt
1	12346	0
2	12347	2458
3	12348	2332
4	12349	630
5	12350	196
6	12352	463
7	12353	20
8	12354	530
9	12355	240
10	12356	1573

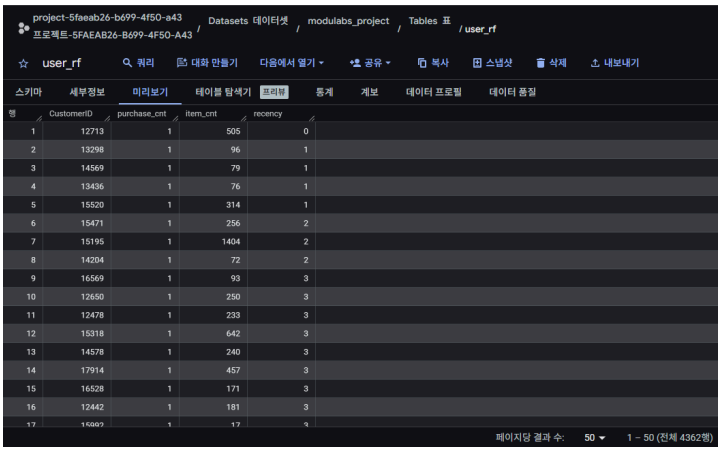
페이지당 결과 수: 50 1 – 50 (전체 4362행)

- **전체 거래 건수 계산와 구매한 아이템의 총 수량 계산의 결과를 합쳐서 `user_rf` 라는 이름의 테이블에 저장하기**

```
CREATE OR REPLACE TABLE `project-5faeab26-b699-4f50-a43.modulabs_project.user_rf` AS

-- (1) 전체 거래 건수 계산
WITH purchase_cnt AS (
    SELECT
        CustomerID,
        COUNT(DISTINCT InvoiceNo) AS purchase_cnt
    FROM `project-5faeab26-b699-4f50-a43.modulabs_project.data`
    GROUP BY 1
),

-- (2) 구매한 아이템 총 수량 계산
item_cnt AS (
    SELECT
        CustomerID,
        SUM(Quantity) AS item_cnt
    FROM `project-5faeab26-b699-4f50-a43.modulabs_project.data`
    GROUP BY 1
)

-- 기존의 user_r에 (1)과 (2)를 통합
SELECT
  pc.CustomerID,
  pc.purchase_cnt,
  ic.item_cnt,
  ur.recency
FROM purchase_cnt AS pc
JOIN item_cnt AS ic
  ON pc.CustomerID = ic.CustomerID
JOIN `project-5faeab26-b699-4f50-a43.modulabs_project.user_r` AS ur
  ON pc.CustomerID = ur.CustomerID;
```

[결과 이미지]



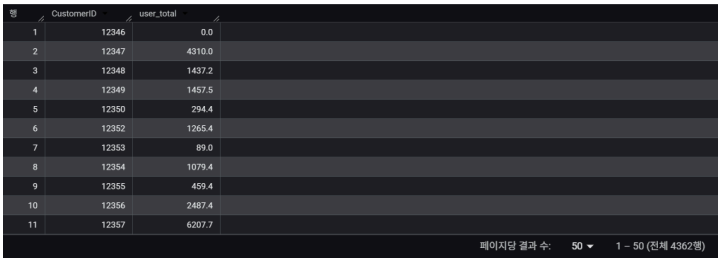


Monetary

- **고객별 총 지출액 계산 (소수점 첫째 자리에서 반올림)**

```
SELECT
    CustomerID,
    ROUND(SUM(Quantity * UnitPrice), 1) AS user_total
FROM `project-5faeab26-b699-4f50-a43.modulabs_project.data`
GROUP BY 1;
```

[결과 이미지]

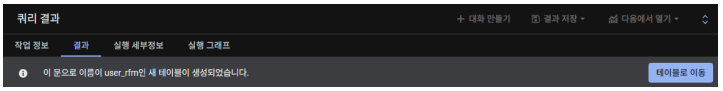


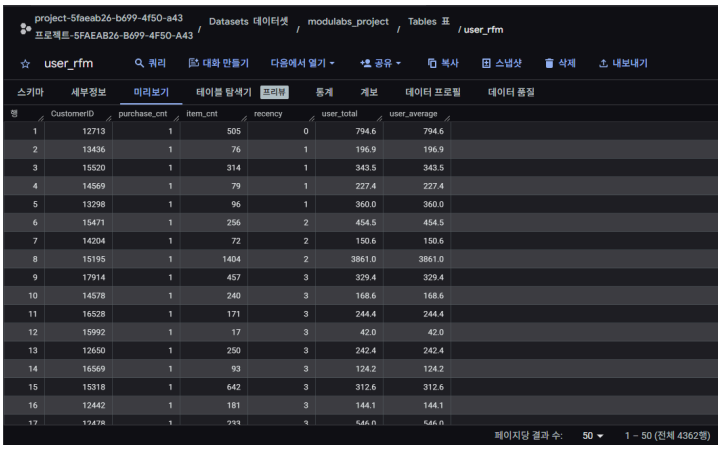
- **고객별 평균 거래 금액 계산**
  - **고객별 평균 거래 금액을 구하기 위해 1) `data` 테이블을 `user_rf` 테이블과 조인(LEFT JOIN) 한 후, 2) `purchase_cnt` 로 나누어서 3) `user_rfm` 테이블로 저장하기**

```
CREATE OR REPLACE TABLE `project-5faeab26-b699-4f50-a43.modulabs_project.user_rfm` AS
SELECT
  rf.CustomerID AS CustomerID,
  rf.purchase_cnt,
  rf.item_cnt,
  rf.recency,
  ut.user_total,
  ROUND(ut.user_total/rf.purchase_cnt, 1) AS user_average
FROM `project-5faeab26-b699-4f50-a43.modulabs_project.user_rf` AS rf
LEFT JOIN (
  -- 고객 별 총 지출액
  SELECT
      CustomerID,
      ROUND(SUM(Quantity * UnitPrice), 1) AS user_total
  FROM `project-5faeab26-b699-4f50-a43.modulabs_project.data`
```

```
  GROUP BY 1
) AS ut
ON rf.CustomerID = ut.CustomerID;
```

[결과 이미지]



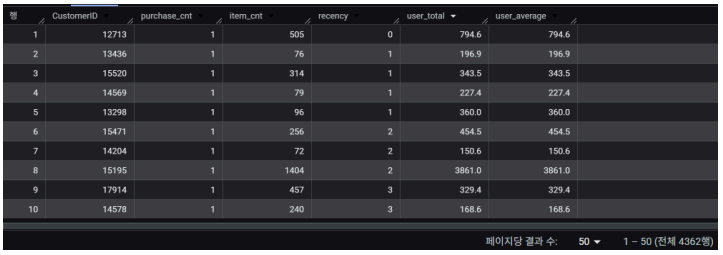


RFM 통합 테이블 출력하기

- **최종 user_rfm 테이블을 출력하기**

```
SELECT *
FROM `project-5faeab26-b699-4f50-a43.modulabs_project.user_rfm`
```

[결과 이미지]



11-8. 추가 Feature 추출

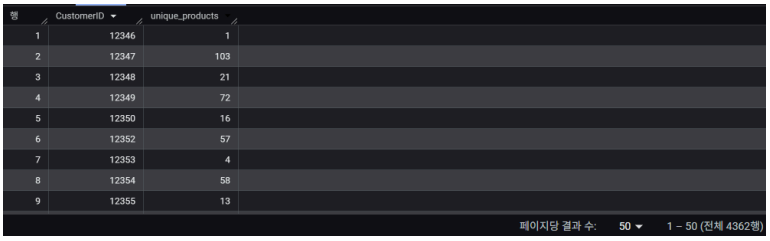
1. 구매하는 제품의 다양성

- **1) 고객 별로 구매한 상품들의 고유한 수를 계산하기**
  2) `user_rfm` 테이블과 결과를 합치기
  3) `user_data` 라는 이름의 테이블에 저장하기

```
CREATE OR REPLACE TABLE `project-5faeab26-b699-4f50-a43.modulabs_project.user_data` AS
WITH unique_products AS (
  SELECT
    CustomerID,
    COUNT(DISTINCT StockCode) AS unique_products
  FROM `project-5faeab26-b699-4f50-a43.modulabs_project.data`
  GROUP BY CustomerID
)
```

```
SELECT ur.*, up.* EXCEPT (CustomerID)
FROM `project-5faeab26-b699-4f50-a43.modulabs_project.user_rfm` AS ur
JOIN unique_products AS up
ON ur.CustomerID = up.CustomerID;
```

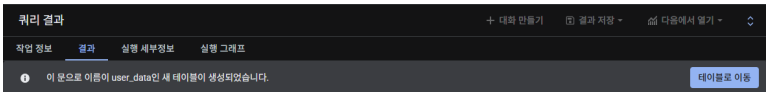
[결과 이미지 - 고객별 구매한 상품의 고유한 수 계산]



행	CustomerID	unique_products
1	12346	1
2	12347	103
3	12348	21
4	12349	72
5	12350	16
6	12352	57
7	12353	4
8	12354	58
9	12355	13

페이지당 결과 수: 50 1 – 50 (전체 4362행)

[결과 이미지 - 'user_data' 새 테이블 생성]



행	CustomerID	purchase_cnt	item_cnt	recency	user_total	user_average	unique_prod...
1	18184	1	60	15	49.8	49.8	1
2	15488	1	72	92	76.3	76.3	1
3	15118	1	1440	134	244.8	244.8	1
4	16344	1	18	158	101.1	101.1	1
5	18133	1	1350	212	931.5	931.5	1
6	17331	1	16	123	175.2	175.2	1
7	15940	1	4	311	35.8	35.8	1
8	18068	1	6	289	101.7	101.7	1
9	13829	1	-12	359	-102.0	-102.0	1
10	16995	1	-1	372	-1.3	-1.3	1
11	16990	1	100	218	179.0	179.0	1
12	14576	1	12	372	35.4	35.4	1
13	16257	1	1	176	21.9	21.9	1
14	17763	1	12	263	15.0	15.0	1
15	13188	1	24	11	99.6	99.6	1
16	16953	1	10	30	20.8	20.8	1

페이지당 결과 수: 50 1 – 50 (전체 4362행)

2. 평균 구매 주기

- **고객들의 쇼핑 패턴을 이해하는 것을 목표 (고객 별 재방문 주기 살펴보기)**
  - **균 구매 소요 일수를 계산하고, 그 결과를 `user_data`에 통합**

```
CREATE OR REPLACE TABLE `project-5faeab26-b699-4f50-a43.modulabs_project.user_data` AS
WITH purchase_intervals AS (
    -- (2) 고객별 구매와 구매사이 기간 평균
    SELECT
        CustomerID,
        CASE WHEN ROUND(AVG(interval_), 2) IS NULL THEN 0 ELSE ROUND(AVG(interval_), 2) END A
S average_interval
    FROM(
    -- (1) 고객별 구매와 구매 사이의 기간 계산
        SELECT
            CustomerID,
            DATE_DIFF(InvoiceDate, LAG(InvoiceDate, 1) OVER (PARTITION BY CustomerID ORDER BY
InvoiceDate), DAY) AS interval_
        FROM `project-5faeab26-b699-4f50-a43.modulabs_project.data`
        WHERE CustomerID IS NOT NULL
    )
    GROUP BY 1
```

```
)

SELECT u.*, pi.* EXCEPT (CustomerID)
FROM `project-5faeab26-b699-4f50-a43.modulabs_project.user_data` AS u
LEFT JOIN purchase_intervals AS pi
ON u.CustomerID = pi.CustomerID;
```

[결과 이미지 - 고객별 구매와 구매 사이 기간 계산]

행	CustomerID	interval_
1	12346	null
2	12346	0
3	12347	null
4	12347	0
5	12347	0
6	12347	0
7	12347	0
8	12347	0
9	12347	0
10	12347	0
11	12347	0
12	12347	0
13	12347	0

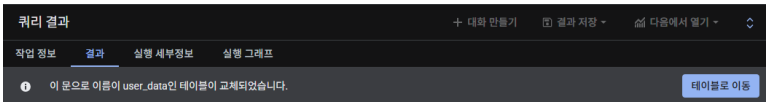
페이지당 결과 수: 50 1 – 50 (전체 399573행)

[결과 이미지 - 고객별 구매와 구매 사이 기간 평균]

행	CustomerID	average_interval
1	12346	0.0
2	12347	2.0
3	12348	10.85
4	12349	0.0
5	12350	0.0
6	12352	3.11
7	12353	0.0
8	12354	0.0
9	12355	0.0
10	12356	5.3
11	12357	0.0
12	12358	9.31
13	12359	1.28
14	12360	1.18

페이지당 결과 수: 50 1 – 50 (전체 4362행)

[결과 이미지 - user_data에 통합]



project-5faeab26-b699-4f50-a43 / Datasets 데이터셋 / modulabs_project / Tables 표 / user_data
프로젝트-5FAEAB26-B699-4F50-A43

user_data 쿼리 대화 만들기 다음에서 열기 공유 복사 스냅샷 삭제 내보내

스키마 세부정보 미리보기 테이블 탐색기 프리뷰 통계 계보 데이터 프로필 데이터 품질

행	CustomerID	purchase_cnt	item_cnt	recency	user_total	user_average	unique_prod...	average_inte...
1	18174	1	50	7	104.0	104.0	1	0.0
2	13135	1	4300	196	3096.0	3096.0	1	0.0
3	16344	1	18	158	101.1	101.1	1	0.0
4	17986	1	10	56	20.8	20.8	1	0.0
5	16144	1	16	246	175.2	175.2	1	0.0
6	14705	1	100	198	179.0	179.0	1	0.0
7	13270	1	200	366	590.0	590.0	1	0.0
8	15668	1	72	217	76.3	76.3	1	0.0
9	14119	1	-2	354	-19.9	-19.9	1	0.0
10	18233	1	4	325	440.0	440.0	1	0.0
11	16995	1	-1	372	-1.3	-1.3	1	0.0
12	13703	1	10	318	99.5	99.5	1	0.0
13	13366	1	144	50	56.2	56.2	1	0.0
14	17925	1	72	372	244.1	244.1	1	0.0
15	16881	1	600	66	432.0	432.0	1	0.0
16	13099	1	288	99	207.4	207.4	1	0.0

페이지당 결과 수: 50 1 – 50 (전체 4362행)

3. 구매 취소 경향성

- **고객의 취소 패턴 파악하기**
  1) 취소 빈도(cancel_frequency) : 고객 별로 취소한 거래의 총 횟수
  2) 취소 비율(cancel_rate) : 각 고객이 한 모든 거래 중에서 취소를 한 거래의 비율
  - **취소 빈도와 취소 비율을 계산하고 그 결과를 `user_data`에 통합하기**
    (취소 비율은 소수점 두번째 자리)

```
CREATE OR REPLACE TABLE `project-5faeab26-b699-4f50-a43.modulabs_project.user_data` AS
WITH TransactionInfo AS (
  SELECT
    CustomerID,
    -- (1) 고객별 전체 거래 횟수
    COUNT(DISTINCT InvoiceNo) AS total_transactions,
    -- (2) 고객별 취소한 거래의 총 횟수
    COUNT(DISTINCT CASE WHEN InvoiceNo LIKE 'C%' THEN InvoiceNo END) AS cancel_frequency
  FROM `project-5faeab26-b699-4f50-a43.modulabs_project.data`
  GROUP BY 1
)
--(3) 각 고객별 총 거래중 취소한 거래 비율 계산
SELECT u.*, t.* EXCEPT(CustomerID), ROUND(cancel_frequency/total_transactions * 100,2) AS can
cel_rate
FROM `project-5faeab26-b699-4f50-a43.modulabs_project.user_data` AS u
LEFT JOIN TransactionInfo AS t
ON u.CustomerID = t.CustomerID
```

[결과 이미지-고객별 전체 거래 횟수 및 취소 거래수]

행	CustomerID	total_transactions	cancel_frequency
1	12346	2	1
2	12347	7	0
3	12348	4	0
4	12349	1	0
5	12350	1	0
6	12352	8	1
7	12353	1	0
8	12354	1	0
9	12355	1	0
10	12356	3	0
11	12357	1	0
12	12358	2	0
13	12359	6	2

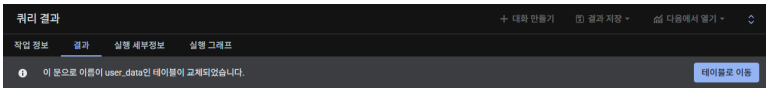
페이지당 결과 수: 50 1 – 50 (전체 4362행)

[결과 이미지-고객별 전체 취소비율 계산 결과]

행	CustomerID	purchase_cnt	item_cnt	recency	user_total	user_average	unique_products	average_interval	total_transactions	cancel_frequency	cancel_rate
1	18174	1	50	7	104.0	104.0	1	0.0	1	0	0.0
2	13135	1	4300	196	3096.0	3096.0	1	0.0	1	0	0.0
3	16344	1	18	158	101.1	101.1	1	0.0	1	0	0.0
4	17986	1	10	56	20.8	20.8	1	0.0	1	0	0.0
5	16144	1	16	246	175.2	175.2	1	0.0	1	0	0.0
6	14705	1	100	198	179.0	179.0	1	0.0	1	0	0.0
7	13270	1	200	366	590.0	590.0	1	0.0	1	0	0.0
8	15668	1	72	217	76.3	76.3	1	0.0	1	0	0.0
9	14119	1	-2	354	-19.9	-19.9	1	0.0	1	1	100.0
10	18233	1	4	325	440.0	440.0	1	0.0	1	0	0.0
11	16995	1	-1	372	-1.3	-1.3	1	0.0	1	1	100.0

페이지당 결과 수: 50 1 – 50 (전체 4362행)

[결과 이미지-user_data에 통합]



행	CustomerID	purchase_cnt	item_cnt	recency	user_total	user_average	unique_prod...	average_inte...	total_transac...	cancel_frequ...	cancel_rate
1	16623	1	39	245	129.5	129.5	5	0.0	1	0	0.0
2	15705	1	62	201	143.1	143.1	6	0.0	1	0	0.0
3	14494	1	92	25	139.6	139.6	7	0.0	1	0	0.0
4	15958	1	33	74	104.4	104.4	7	0.0	1	0	0.0
5	17698	1	93	260	155.0	155.0	8	0.0	1	0	0.0
6	16658	1	40	368	123.2	123.2	8	0.0	1	0	0.0
7	17947	1	168	37	176.7	176.7	9	0.0	1	0	0.0
8	18278	1	66	73	173.9	173.9	9	0.0	1	0	0.0
9	18066	1	108	75	318.0	318.0	10	0.0	1	0	0.0
10	12441	1	120	366	155.6	155.6	10	0.0	1	0	0.0
11	15885	1	109	93	349.3	349.3	16	0.0	1	0	0.0
12	15193	1	144	60	499.3	499.3	19	0.0	1	0	0.0
13	15149	1	128	133	520.8	520.8	23	0.0	1	0	0.0
14	17154	1	490	75	329.3	329.3	23	0.0	1	0	0.0
15	15314	1	313	199	328.0	328.0	24	0.0	1	0	0.0
16	13603	1	164	26	164.3	164.3	27	0.0	1	0	0.0

- **다양한 컬럼들을 활용하여 고객의 구매 패턴과 선호도를 보다 심층적으로 이해할 수 있도록 최종적으로 `user_data`를 출력하기**

```
SELECT *
FROM `project-5faeab26-b699-4f50-a43.modulabs_project.user_data`
```

[결과 이미지]

행	CustomerID	purchase_c...	item_cnt	recency	user_total	user_average	unique_products	average_interval	total_transactions	cancel_frequency	cancel_rate
1	16623	1	39	245	129.5	129.5	5	0.0	1	0	0.0
2	15705	1	62	201	143.1	143.1	6	0.0	1	0	0.0
3	14494	1	92	25	139.6	139.6	7	0.0	1	0	0.0
4	15958	1	33	74	104.4	104.4	7	0.0	1	0	0.0
5	17698	1	93	260	155.0	155.0	8	0.0	1	0	0.0
6	16658	1	40	368	123.2	123.2	8	0.0	1	0	0.0
7	17947	1	168	37	176.7	176.7	9	0.0	1	0	0.0
8	18278	1	66	73	173.9	173.9	9	0.0	1	0	0.0
9	18066	1	108	75	318.0	318.0	10	0.0	1	0	0.0
10	12441	1	120	366	155.6	155.6	10	0.0	1	0	0.0
11	15885	1	109	93	349.3	349.3	16	0.0	1	0	0.0
12	15193	1	144	60	499.3	499.3	19	0.0	1	0	0.0
13	15149	1	128	133	520.8	520.8	23	0.0	1	0	0.0

페이지당 결과 수: 50 1 – 50 (전체 4362행)

회고

[회고 내용을 작성해주세요]

Keep :
- DELETE 나 REPLACE 명령어 실행 전 데이터 확인
- 오류 메시지를 읽으면서 논리를 다시 세워보는 자세
- 느리지만 문제를 다시 읽으면서 문제의 의도를 파악하여 쿼리를 수정한 점
- 아직은 능숙하지 않은 것이 당연하니까, 단계별로 쿼리를 나눠서 검증 후 진행한 것 (돌다리도 두드려 보자!)

Problem :
- DELETE로 데이터 삭제시, 원상복구 방법 (최근 1시간 전 데이터 확인, 백업데이터 작성, REPLACE 처리)
- SELECT 문 작성시 집계함수와 일반 컬럼을 함께 작성시 빈번한 오류 발생
- 윈도우함수에서의 OVER() : OVER() 와 OVER(그룹화 기준 PARTITION BY col ORDER BY col)과의 차이
- DISTINCT 위치, CASE WHEN ~ THEN ... ELSE - END, UPPER() 등 함수의 문법에 대한 미숙함
- 서브쿼리 중첩구조 : 서브쿼리 실행 후 메인쿼리 실행

Try :
- 그룹함수, 집계함수, 순위함수, 행순서 함수, 임시테이블 등 다양한 활용 및 예제를 통한 연습 필요
- 쿼리 작성 전, 머리 속으로 예상되지 않는 다면, 결과에 대해 표로 작성해서 구체화 해보기
- DELETE로 테이블 복구시 사용했던 TimeTravel 와 같은 BigQuery 기능 확인해보기